Σενάρια φλοιού
(Shell scripts)
C

Διεργασίες

- Κάθε διεργασία κάνοντας exit επιστρέφει έναν κωδικό εξόδου.
  - Είτε με exit(0), exit(1) etc….
  - Είτε με return(κωδικός) από την συνάρτηση main()
- Όταν μια διεργασία κάνει exit (ή τερματίζεται για οποιοδήποτε λόγο) τότε πηγαίνει το αντίστοιχο signal στην γονική διεργασία (SIGCHLD)

- Η γονική διεργασία πρέπει να κάνει “collect” το “παιδί” που τερματίστηκε.
- Μέχρι ο γονέας να κάνει “collect”, η τερματισμένη διεργασία παραμένει στον πίνακα διεργασιών με status: Zombie.
- Το “collect” γίνεται με τις system calls:

  wait , waitpid
- Η wait επιστρέφει στον γονέα τον “exit code” του παιδιού που τερματίστηκε.
- Όταν η γονική διεργασία είναι το shell, χρησιμοποιεί το “exit code” του παιδιού που τερματίστηκε.

Exit codes

- Η μεταβλητή $? «γεμίζει» με το exit code της θυγατρικής διεργασίας.
- Πχ:

```
ls –l /tmp > /dev/null
echo $?
0
ls –l /tmdddp > /dev/null
ls: /tmdddp: Δεν υπάρχει τέτοιο αρχείο ή
  κατάλογος
echo $?
1
```

Exit codes

- Μια γραμμή με εντολές είναι μια λογική συνθήκη.
- || → OR , && → AND, ! → NOT
- Πχ:

```
wc -l gfgfg && echo OK
wc: gfgfg: Δεν υπάρχει τέτοιο αρχείο ή κατάλογος

wc -l p_dblab3.txt && echo OK
     24 p_dblab3.txt
OK

wc -l gfgfg || echo NOT OK
wc: gfgfg: Δεν υπάρχει τέτοιο αρχείο ή κατάλογος
NOT OK

wc -l p_dblab3.txt ||  echo NOT OK
     24 p_dblab3.txt
```

- Όταν υπάρχει OR και η πρώτη εντολή εκτελεστεί σωστά δεν χρειάζεται να εκτελεστεί η δεύτερη.
- Όταν υπάρχει AND η δεύτερη εντολή εκτελείται μόνον όταν εκτελέστηκε η πρώτη σωστά.

Η εντολή if

```
if expression;      then
    commands;
  elif expression; then
    commands;
  elif expression; then
    commands;
  else
    commands;
fi
```

```
if ls –l “$dir”; then
        echo “OK”
else
        echo “NOT OK”
fi
```

```
if cat file
   then
            echo "file exists“;
   else
            echo "file does not exist“;
   fi
```

Η εντολή test

Η εντολή test μας βοηθάει να κάνουμε διάφορους ελέγχους. Η test κάνει exit 0 (true) αν η λογική έκφραση είναι αληθής, και exit 1 (false) αν η λογική έκφραση είναι ψευδής

string comparison

Expression	***Description***
-z *string*	True if *string* is empty.
-n *string*	True if *string* is not empty.
string1 = *string2*	True if *string1* equals *string2*.
string1 != *string2*	True if *string1* does not equal *string2*.

Η εντολή test

Integer comparison

Expression	*Description*
INTEGER1 -eq INTEGER2	INTEGER1 is equal (=) to INTEGER2
INTEGER1 -ge INTEGER2	INTEGER1 is greater than or equal (>=) to INTEGER2
INTEGER1 -gt INTEGER2	INTEGER1 is greater than (>) INTEGER2
INTEGER1 -le INTEGER2	INTEGER1 is less than or equal (>=) to INTEGER2
INTEGER1 -lt INTEGER2	INTEGER1 is less than (>) INTEGER2
INTEGER1 -ne INTEGER2	INTEGER1 is not equal (!=) to INTEGER2

Η εντολή test

File system check

Expression	***Description***
-d *file*	True if *file* is a directory.
-e *file*	True if *file* exists.
-f *file*	True if *file* exists and is a regular file.
-L *file*	True if *file* is a symbolic link.
-r *file*	True if *file* is a file readable by you.
-w *file*	True if *file* is a file writable by you.
-x *file*	True if *file* is a file executable by you.
-s file	true if file exists and has a size greater than zero.
file1 -nt *file2*	True if *file1* is newer than (according to modification time) *file2*
file1 -ot *file2*	True if *file1* is older than *file2*

Η εντολή test

Logical operators

Expression	*Description*
!	unary negation operator.
-a	binary and operator.
-o	binary or operator (-a has higher precedence than -o).
(expr)	parentheses for grouping

```
bash-2.05a$ test 5 -ge 7
bash-2.05a$ echo $?
1
bash-2.05a$ test 5 -le 7
bash-2.05a$ echo $?
0
```

test

```
if test -f ~/.bash_profile ; then
    echo "You have a .bash_profile."
else
    echo "Yikes! You have no .bash_profile!"
fi
```

test

```
echo " give a number < 10 "
read a
if test "$a" -ge 10 ; then
    echo "You gave wrong number"
else
    echo "OK. You gave $a"
fi
```

[= test

```
echo " give a number < 10 "
read a
if [ "$a" -ge 10 ]; then
    echo "You gave wrong number"
else
    echo "OK. You gave $a"
fi
```

- Η [είναι εντολή και είναι ισοδύναμη με την εντολή test.
- Γι' αυτό και χρειάζονται κενά μεταξύ του ονόματος της εντολής και των παραμέτρων, καθώς επίσης και μεταξύ των παραμέτρων μεταξύ τους
- Η εντολή [περιμένει η τελευταία παράμετρος που θα της δώσουμε να είναι το string] (για λόγους συμμετρίας και ομορφιάς), αλλιώς μας δίνει μήνυμα λάθους

Έλεγχος αλφαριθμητικών (τελεστής =)

```
#!/bin/bash
number=1
if [ $number = "1" ]; then
   echo "Number equals 1"
else
   echo "Number does not equal 1"
fi
```

```
if [ “$NAME” = “John Doe” -a “$VAL” –gt  5000 ]
        then
                echo “ok”
        else
                echo “not ok”
   fi
```

Παράδειγμα

```
#!/bin/bash
number=0

echo -n "Enter a number > "
read number

echo "Number is  $number"
if [ $(( $number % 2)) -eq 0 ]; then
   echo "Number is even"
else
   echo "Number is odd"
fi
```

Χρήση των εντολών if, test

```
#!/bin/bash
echo -n "Enter a filename to check ==>";
read filename

if test ! -r  $filename
then
  echo "unable to read the file $filename"
  exit 1
fi
```

Παράδειγμα με if και test

```
if test -f $filename
then
  echo $filename is a regular file
elif test -d $filename
then
  echo $filename is a directory
elif [ -h $filename ]
then
  echo $filename is a symbolic link
elif [ -c $filename ]
then
  echo $filename is a character special file
fi
```

- Η εντολή while

  while συνθήκη;
  do
  commands
  done

Παράδειγμα

```
#!/bin/bash
number=0
while [ $number -lt 10 ];
do
   echo "Number = $number"
   number=$((number + 1))
done
```

Η εντολή until

```
#!/bin/bash
number=0
until [ $number -ge 10 ];
do
    echo "Number = $number“
    sleep 1
    number=$((number + 1))
done
```

Παρατηρήσεις

- Όταν χρησιμοποιούμε test commands ([…]) πρέπει να δίνουμε ιδιαίτερη προσοχή στα κενά ανάμεσα στις παραμέτρους της εντολής, διαφορετικά θα υπάρξουν λανθασμένα αποτελέσματα.
- Για τον έλεγχο των αποτελεσμάτων που προκύπτουν κατά την εκτέλεση μιας εντολής μπορούμε να χρησιμοποιούμε την echo $? που επιστρέφει το αποτέλεσμα της αμέσως προηγούμενης εντολής που έχει εκτελεστεί (0=true, !0=false)

Εντολή shift

```
#!/bin/sh
#filename=ss
while [ $# -gt 0 ] ; do
        echo "param = $1, num = $#"
        shift 1
done
```

bash-2.05a$ ./ss one two 3 four 5 six
param = one, num = 6
param = two, num = 5
param = 3, num = 4
param = four, num = 3
param = 5, num = 2
param = six, num = 1
bash-2.05a$

Η εντολή case

```
case value in
   pattern 1) command
              anothercommand
                ;;
   pattern 2) command
                anothercommand
                ;;
esac
```

```
echo –n "Your Answer: "
read ANSWER
case “$ANSWER” in
Y* | y*) ANSWER="YES" ;;
N* | n*) ANSWER="NO" ;;
*) ANSWER="MAYBE" ;;
esac
echo $ANSWER
```

Παράδειγμα στην εντολή case

```
echo -n "Enter the name of an animal: "
read ANIMAL
echo -n "The $ANIMAL has "
case $ANIMAL in
  horse | dog | cat) echo -n "four";;
  man | kangaroo ) echo -n "two";;
  *) echo -n "an unknown number of";;
esac
echo " legs."
```

Εντολή shift

```
bash-2.05a$ ./myls  -d
You have set the params:
long_list=0
hidden_on=0
dirs_only=1
file=
bash-2.05a$ ./myls  -d -f ff
You have set the params:
long_list=0
hidden_on=0
dirs_only=1
file=ff
bash-2.05a$ ./myls  -d -l -f ff
You have set the params:
long_list=1
hidden_on=0
dirs_only=1
file=ff
bash-2.05a$ ./myls -d -e -l
Unknown param -e
Valid params are: [-l] [-a] [-d] [-f file]
bash-2.05a$
```

```
#!/bin/sh
#filename=myls
long_list=0
hidden_on=0
dirs_only=0
file=""
while [ $# -gt 0 ] ;
do
        case "$1" in
          -l) long_list=1;;
          -a) hidden_on=1;;
          -d) dirs_only=1;;
          -f) shift 1; file="$1";;
          *) echo "Unknown param $1";
             echo "Valid params are: [-l] [-a] [-d] [-f file]"
             exit 1
             ;;
         esac
         shift 1
done
echo "You have set the params:
long_list=$long_list
hidden_on=$hidden_on
dirs_only=$dirs_only
file=$file"
```

Απόδοση τιμής σε μεταβλητή με χρήση ``

```
# get the size of the file

filesize=`ls -ld $filename | cut -c31-41`

echo file size is $filesize

if [ $filesize -le 100 ]
then
  echo $filename is less than 100 bytes
elif [ ($filesize -gt 100 ) -a ( $filesize -lt 10000 ) ]
then
  echo $filename is greater than 100 bytes but less than
    10000 bytes
fi
```

Απόδοση τιμής σε μεταβλητή με χρήση **$( )**

```
#!/bin/bash
filesize=$(ls -l $1  |  cut -c31-41)
echo filesize of $1 is $filesize
```

Η χρήση των $( ) είναι ισοδύναμη με αυτή των ` `

read με ανακατεύθυνση

- read a < file1
- Η read διαβάζει μια γραμμή από την κανονική είσοδο (stdin)
- Εάν το file1 δεν έχει δεδομένα, τότε η read θα επιστρέψει false, αλλιώς θα διαβάσει μια γραμμή και θα την αποθηκεύσει στην μεταβλητή a.

read με αλλαγή κανονικής εισόδου

```
#!/bin/sh
#file: reader.sh
while read a; do
        echo "diavasa: $a";
done
```

```
$more file3
line 1
deyterh grammh
third line
grammh 4
$./reader.sh < file3
diavasa: line 1
diavasa: deyterh grammh
diavasa: third line
diavasa: grammh 4
```

read με αλλαγή κανονικής εισόδου

```
#!/bin/sh
#file: reader3.sh
echo -n "Give a filename:"
read file

while read a < "$file"; do
        echo "diavasa: $a";
        sleep 1
done
```

```
$more file3
line 1
deyterh grammh
third line
grammh 4
$./reader.sh < file3
Give a filename:file3
diavasa: line 1
diavasa: line 1
diavasa: line 1
diavasa: line 1
diavasa: line 1
^C
$
```

Blocks εντολών

```
$cat file3
line 1
deyterh grammh
third line
grammh 4
$./readtest2
line 1
$./readtest3
line 1
$./readtest1
deyterh grammh
$./readtest4
```

```
#!/bin/sh
#readtest2
read b <file3
echo $b

#!/bin/sh
#readtest3
read a <file3
read b <file3
echo $b

#!/bin/sh
#readtest1
(read a; read b; echo $b) <file3

#!/bin/sh
#readtest4
(read a; read b) <file3
echo $b
```

Blocks εντολών

```
bash-2.05a$ ps
        PID TTY      TIME CMD
    7406289 ttyq4    0:00 bash
    7334303 ttyq4    0:00 ps
bash-2.05a$ (read a; read b ; echo $b; ps)<file3
deyterh grammh
        PID TTY      TIME CMD
    7433307 ttyq4    0:00 bash
    7406289 ttyq4    0:00 bash
    7394409 ttyq4    0:00 ps
bash-2.05a$ echo $b

bash-2.05a$
```

Όταν ομαδοποιούμε εντολές με (), δημιουργείται ένα νέο shell που εκτελεί τις εντολές

Η μεταβλητή b έχει χαθεί

Blocks εντολών

```
bash-2.05a$ ps
        PID TTY      TIME CMD
    7406289 ttyq4    0:00 bash
    7342217 ttyq4    0:00 ps
bash-2.05a$ b=TEST
bash-2.05a$ echo $b
TEST
bash-2.05a$ (read a; read b ; echo $b; ps) < file3
deyterh grammh
        PID TTY      TIME CMD
    7368432 ttyq4    0:00 ps
    7406289 ttyq4    0:00 bash
    7422665 ttyq4    0:00 bash
bash-2.05a$ echo $b
TEST
```

Η μεταβλητή b έχει διατηρήσει την παλιά τιμή

Blocks εντολών

```
#!/bin/sh
#file: reader5.sh
b=1
echo -n "Give a filename:"
read file
while read a; do
        echo "diavasa $b: $a";
        sleep 1
        b=`expr $b + 1`
done < "$file"
echo $b
```

```
$more file3
line 1
deyterh grammh
third line
grammh 4
aetos_test_98_$./reader.sh
Give a filename:file3
diavasa 1: line 1
diavasa 2: deyterh grammh
diavasa 3: third line
diavasa 4: grammh 4
b=5
$
```

Η εντολή set

- Παίρνει σαν ορίσματα οποιαδήποτε strings και τα «μοιράζει» στις μεταβλητές $1,$2,....

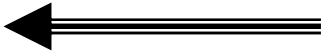
```
bash-2.05a$ set a b "third param"
bash-2.05a$ echo $1
a
bash-2.05a$ echo $2
b
bash-2.05a$ echo $3
third param
```

- **Ο default διαχωριστής που θεωρεί η set είναι το space και tab.**

Η εντολή set

- Μπορούμε να δώσουμε στην set όρισμα μια τιμή μεταβλητής (χωρίς όμως εισαγωγικά)

```
bash-2.05a$ var="a b third"
bash-2.05a$ set $var
bash-2.05a$ echo $1
a
bash-2.05a$ echo $2
b
bash-2.05a$ echo $3
third
```



ΟΧΙ set “$var”

Η εντολή set

- Η εντολή set κάνει τον διαχωρισμό με βάση την τιμή της μεταβλητής IFS (Internal Field Separator)

```
bash-2.05a$ var="a:b:third"
bash-2.05a$ IFS=:
bash-2.05a$ set $var
bash-2.05a$ echo $1
a
bash-2.05a$ echo $2
b
bash-2.05a$ echo $3
third
```



ΟΧΙ set “$var”

- Ένα script που βρίσκει τα home directories όλων των χρηστών:

```
#!/bin/sh

IFS=":"
while read a; do
        set $a
        echo "user=$1 , home=$6"
done < /etc/passwd
```